

MU5MAI04

Approfondissement C++

Daphné Giorgi

Table of contents

Description	4
Objectifs	4
Evaluation	4
I GIT ET GITHUB	5
Objectifs	6
1 Description	7
2 Concepts de base	8
2.0.1 Espaces de travail	8
2.0.2 Commandes	8
3 Les remote	10
4 Install et configuration	11
4.0.1 Install de git	11
4.0.2 Configuration	11
5 Initialiser un projet	13
5.0.1 Initialisation d'un répertoire	13
5.0.2 Clone d'un projet distant	14
6 Travail en local	15
7 Travail avec remote	16
8 Travail en équipe	17
9 Workflow	18
10 .gitignore	19
11 Evaluation	21

II	MAKEFILE	22
	Objectifs	23
12	Compilation	24
	12.0.1 Compilation de plusieurs fichiers	24
13	Makefile	25
	13.0.1 La syntaxe	25
	13.0.2 Un projet “hello world”	25
14	Makefile enrichi	28
15	Variables	29
16	Cmake	32
	16.0.1 Autre	32

Description

Ce cours concerne les étudiants IMPA et IMPC du [Master Mathématiques et Applications](#).

Emplois du temps : [EdT IMPA](#) et [EdT IMPC](#).

Moodle : [MU5MAI04](#)

Organisation : Chaque séance mélangera cours et travaux pratiques.

Prérequis : Fondamentaux de C++ par V. Lemaire.

Objectifs

Savoir développer un projet en C++ en utilisant les outils pour le développement tels que

- gestion de version
- debug
- test
- intégration continue
- notebook

Evaluation

La note finale sera donnée sur la base d'un travail à effectuer en binôme (deux personnes), au cours du S1, pendant les séances du cours.

On rappelle les règles de fonctionnement des binômes :

- pas de binôme mixte (apprenti - non apprenti)
- pas plus de trois projets au cours de l'année avec la même personne.

Chaque binôme devra rendre un **lien vers un dépôt git** contenant l'historique des commit du projet (et donc aussi la version finale du code). On ne vous demande pas de rapport. Une discussion d'environ **trente minutes** aura lieu pour m'aider à juger ce que vous avez fait. La date de rendu du dépôt git sera une semaine avant la soutenance.

Part I

GIT ET GITHUB

Objectifs

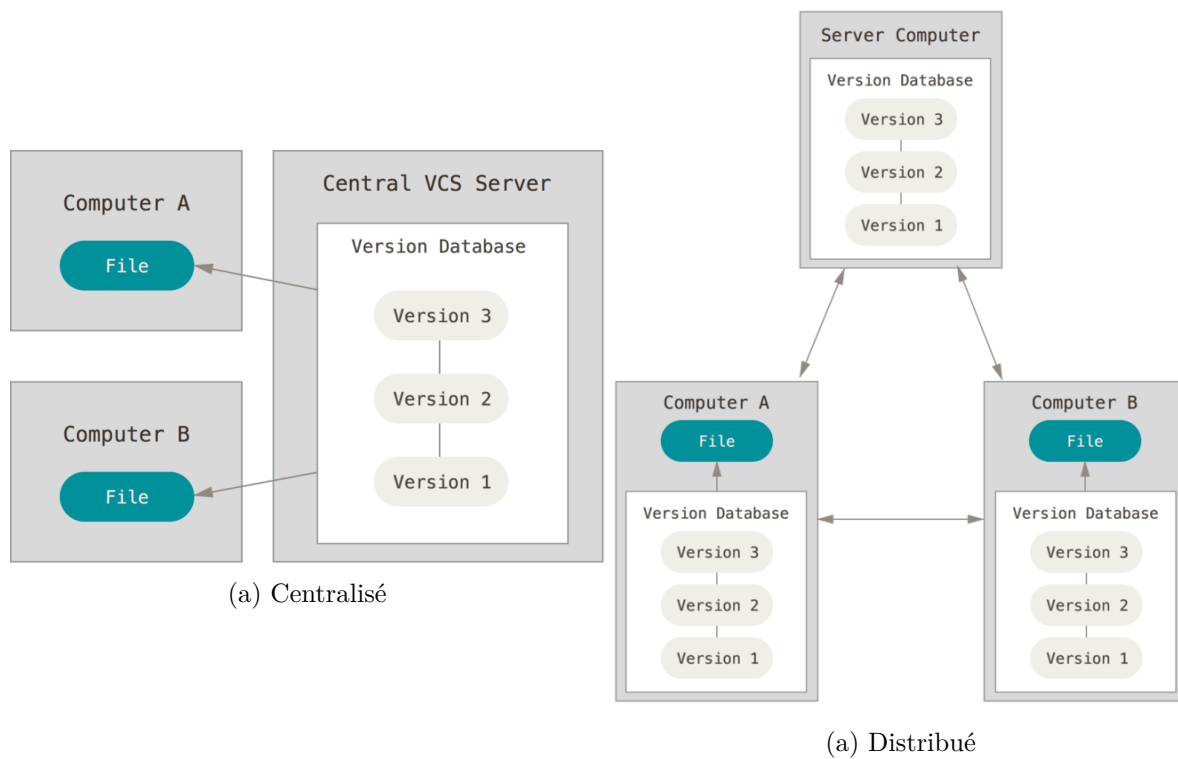
- Gérer un projet C++ avec Git
- Travailler en équipe avec GitHub
- Utiliser les branches, PR et git hooks

Sommaire :

[Chapter 1](#) [Chapter 2](#) [Chapter 3](#) [Chapter 4](#) [Chapter 5](#) [Chapter 6](#) [Chapter 7](#) [Chapter 8](#) [Chapter 9](#)
[Chapter 10](#)

1 Description

`git` est un système de contrôle de version distribué, libre, conçu pour être rapide et efficace.



Il est utilisé pour

- tracer les modifications dans un projet,
- identifier par qui ont été faites ces modifications,
- coder en équipe.

2 Concepts de base

2.0.1 Espaces de travail

Workspace/dossier de travail : Le dossier courant, ce que je vois lorsque j'ouvre un fichier et je travaille dessus.

Repository/Dépôt : Un dossier dans lequel Git enregistre le projet et son historique.

- **Local repository/dépôt local** : Endroit sur mon ordinateur où `git` stocke l'information sur l'historique des fichiers contenus dans le projet.
- **Remote repository/dépôt distant** : Endroit où est centralisé l'historique de mon projet, souvent sur un cloud de type GitHub/GitLab/..., et où tous les collaborateurs peuvent contribuer.

Staging area/index : Espace de stockage temporaire de l'historique `git`, endroit de *pre-commit*.

2.0.2 Commandes

Navettes qui permettent de se déplacer entre les espaces.

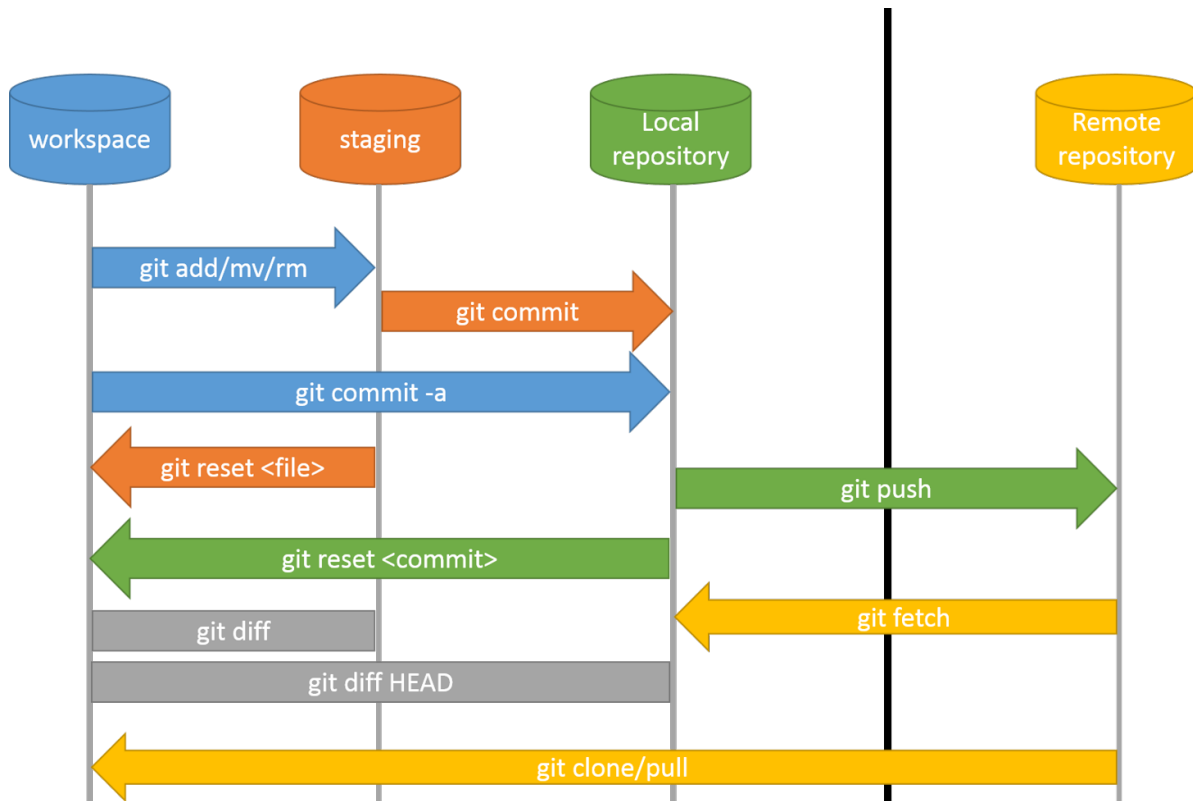


Figure 2.1: Répertoires locaux et distants

clone/Cloner : Copie d'un dépôt distant sur votre ordinateur.

stage/Index : Indiquer à Git les modifications que vous souhaitez enregistrer ensuite.

commit : Sauvegarde des modifications effectuées, ajout d'une ligne dans l'historique du projet.

branch/Branche : Travail sur différentes versions ou fonctionnalités en même temps.

merge/Fusionner : Combiner des modifications provenant de différentes branches.

pull/Récupérer : Obtenir les dernières modifications d'un dépôt distant.

push/Pousser : Envoyer vos modifications à un dépôt distant.

3 Les remote

[GitHub](#) n'est pas la même chose que `git`.

GitHub, tout comme GitLab, Bitbucket, ou autres, est un hébergeur de répertoires `git`.

Exercice

Création d'un compte sur [GitHub](#).

4 Install et configuration

4.0.1 Install de git

Vous pouvez télécharger Git gratuitement à partir de git-scm.com.

- **Windows** : Téléchargez et exécutez le programme d'installation.
Cliquez sur « Suivant » pour accepter les paramètres recommandés.
Cela installera Git et Git Bash.

- **macOS** : Si vous utilisez Homebrew, ouvrez Terminal et tapez

```
brew install git
```

Ou téléchargez le fichier `.dmg` et glissez Git dans votre dossier **Applications**.

- **Linux** : Ouvrez votre terminal et utilisez votre gestionnaire de paquets.

Par exemple, sur Ubuntu :

```
sudo apt-get install git
```

4.0.2 Configuration

La toute première fois, il est nécessaire de configurer `git` pour lui indiquer qui nous sommes.

Cette configuration peut être changée à tout moment et affecte seulement le nom et le mail qui apparaissent dans le commits.

User name

Tapez la commande

```
git config --global user.name "Prénom Nom"
```

email

Tapez la commande

```
git config --global user.email "monadresse@exemple.com"
```

Raccourcis

Parfois, taper les commandes `git` peut paraître long pour une utilisation régulière.

Il est possible de configurer des raccourcis, par exemple, en tapant

```
git config --global alias.ci commit
```

la commande `git ci` sera équivalente à `git commit`.

Pour ouvrir le fichier de config en mode édition on peut également taper

```
git config --global --edit
```

Exercice

Installation et configuration de git en local.

5 Initialiser un projet

On peut démarrer un dépôt `git` de deux manières :

- transformer un répertoire existant en dépôt `git`,
- cloner un répertoire distant.

Dans les deux cas, un répertoire local sur lequel travailler sera présent sur votre machine, on le reconnaît par la présence, à sa racine, d'un sous-répertoire `.git`.

5.0.1 Initialisation d'un répertoire

Partons la structure de code suivante :

```
CalculatorProject/  
  src/  
    Calculator.cpp  
  include/  
    Calculator.h  
  README.md
```

avec

```
include/Calculator.h
```

```
#ifndef CALCULATOR_H  
#define CALCULATOR_H  
  
class Calculator {  
public:  
    double add(double a, double b);  
    double subtract(double a, double b);  
    double multiply(double a, double b);  
    double divide(double a, double b);  
};  
  
#endif
```

src/Calculator.cpp

```
#include "Calculator.h"
#include <stdexcept>

double Calculator::add(double a, double b) {
    return a + b;
}

double Calculator::subtract(double a, double b) {
    return a - b;
}

double Calculator::multiply(double a, double b) {
    return a * b;
}

double Calculator::divide(double a, double b) {
    if (b == 0) throw std::invalid_argument("Division by zero");
    return a / b;
}
```

Pour initialiser ce projet, pour commencer à le versionner avec git, il suffit de taper :

```
cd CalculatorProject
git init
```

5.0.2 Clone d'un projet distant

```
git clone https://github.com/ton-org/ton-projet.git
cd ton-projet
git checkout -b develop
```

Exercice

- Création d'un projet.
- clone d'un projet.

6 Travail en local

Exercice

- Initialiser un projet
- Commande `status` :
 - avant et après création ou édition d'un fichier
 - avant et après ajout d'un fichier à l'index ou à l'historique
- Vérification des différences entre les changements locaux et l'historique : commande `diff --staged`
- Commande `add`
- Commande `commit`
- Commande `log`

7 Travail avec remote

Exercice

- Création d'un projet sur GitHub
- Commande `remote`
- Mise en ligne
- Commande `push`

8 Travail en équipe

Exercice

Travail en binôme.

- Récupération du travail d'un collègue
- Commande `clone`
- Commande `pull` : `fetch+ merge`
- Gestion d'un conflit

9 Workflow

Exercice

Travail en binôme.

- Commande `branch`
- Commande `switch`
- Commande `checkout`

10 .gitignore

Le fichier `.gitignore` est un simple fichier texte placé dans le répertoire racine de votre dépôt Git. Il contient une liste de fichiers et de répertoires à ignorer, comme les fichiers temporaires, les fichiers générés par le système, les fichiers de configuration, ou ceux liés à un build.

Il permet de garder un historique propre et de ne pas intégrer dans le projet les fichiers superflus.

```
##### Build files
build/

##### Log Files
*.log

##### C++
# Prerequisites
*.d

# Compiled Object files
*.slo
*.lo
*.o
*.obj

# Precompiled Headers
*.gch
*.pch

# Compiled Dynamic libraries
*.so
*.dylib
*.dll

# Fortran module files
*.mod
*.smod
```

```
# Compiled Static libraries
*.lai
*.la
*.a
*.lib

# Executables
*.exe
*.out
*.app

#### VisualStudioCode
.vscode/*
!.vscode/settings.json
!.vscode/tasks.json
!.vscode/launch.json
!.vscode/extensions.json
*.code-workspace

# Local History for Visual Studio Code
.history/
```

11 Evaluation

Les critères suivants seront évalués :

- Le code a été rendu via un **dépôt git**
- Les commits sont **unitaires**
- Les noms des commits sont **explicites**
- Le nombre de commits est **équilibré entre les membres** de l'équipe

Part II

MAKEFILE

Objectifs

- Makefile
- Compiler avec cmake

Sommaire :

[Chapter 12](#) [Chapter 13](#) [Chapter 14](#) [Chapter 15](#) [Chapter 16](#)

12 Compilation

Une première façon de compiler les fichiers d'un projet est d'appeler directement le compilateur (ici `g++`) en ligne de commande.

Pour compiler en ligne de commande un seul fichier `hello.cpp` on peut utiliser

```
g++ -o hello hello.cpp
```

On peut aussi passer par l'étape intermédiaire de création d'un fichier objet.

```
g++ -c hello.cpp
g++ -o hello hello.o
```

Ceci génère un fichier supplémentaire `hello.o`.

12.0.1 Compilation de plusieurs fichiers

On peut compiler un code de plusieurs fichiers avec la commande

```
g++ -o programme un.cpp deux.cpp trois.cpp quatre.cpp
```

Cependant, avec cette option, il faut recompiler tous les fichiers à chaque modification d'un seul fichier. Ceci n'est pas une bonne option dès que le temps de compilation n'est pas négligeable. On va donc commencer par créer un fichier objet (`.o`) pour chaque fichier source (`.cpp`), puis on finira la compilation à partir de ces fichiers objets par l'étape d'édition de liens.

Note

Demo

Pour simplifier l'étape de compilation et indiquer au compilateur quelles parties d'un programme doivent être recompilées, ainsi que les dépendances entre fichiers, on pourra utiliser un Makefile ou bien CMake.

13 Makefile

Le plus simple des **Makefile** : copier dans un fichier **Makefile** le texte suivant

```
hello:
    echo "Hello, World"
```

i Attention

Un fichier Makefile doit être indenté avec des TAB et pas des espaces, sinon la commande **make** ne marchera pas.

et dans un terminal lancer la commande **make**.

13.0.1 La syntaxe

Un makefile est constitué d'un ensemble de *règles*. Une règle s'écrit de la façon suivante :

```
cible: dependance
    commandes
```

où chaque commande est précédée par une tabulation.

L'évaluation d'une règle se fait en plusieurs étapes :

- Les dépendances sont analysées, si une dépendance est la cible d'une autre règle du Makefile, cette règle est à son tour évaluée.
- Lorsque l'ensemble des dépendances est analysé et si la cible ne correspond pas à un fichier existant ou si un fichier dépendance est plus récent que la règle, les différentes commandes sont exécutées.

13.0.2 Un projet “hello world”

Nous allons utiliser un projet classique “Hello world” découpé en trois fichiers :

Le Makefile minimal du projet est

Listing 13.1 hello.h

```
#ifndef H_HELLO
#define H_HELLO

void Hello(void);

#endif
```

Listing 13.2 hello.cpp

```
#include <iostream>

void Hello(void)
{
    std::cout << "Hello world!" << std::endl;
}
```

i Exercice

Construire le projet minimal et commenter.

Les fichiers objets qui dépendent de fichiers sources qui n'ont pas été modifiés ne sont pas recompilés (à condition que l'on ait bien défini la liste de dépendances).

Listing 13.3 main.cpp

```
#include "hello.h"

int main(void)
{
    Hello();
    return 0;
}
```

Listing 13.4 Makefile

```
hello: hello.o main.o
    g++ -o hello hello.o main.o

hello.o: hello.cpp
    g++ -o hello.o -c hello.cpp -W -Wall -ansi -pedantic

main.o: main.cpp hello.h
    g++ -o main.o -c main.cpp -W -Wall -ansi -pedantic
```

14 Makefile enrichi

Plusieurs cas ne sont pas gérés dans l'exemple précédent :

- Un tel Makefile ne permet pas de générer plusieurs exécutables distincts.
- Les fichiers intermédiaires restent sur le disque dur même lors de la mise en production.
- Il n'est pas possible de forcer la régénération intégrale du projet.

Ces différents cas conduisent à l'écriture de règles complémentaires :

- **all** : généralement la première du fichier, elle regroupe dans ces dépendances l'ensemble des exécutables à produire.
- **clean** : elle permet de supprimer tous les fichiers intermédiaires.
- **mrproper** : elle supprime tout ce qui peut être régénéré et permet une reconstruction complète du projet.

En ajoutant ces règles complémentaires, le Makefile devient donc :

Listing 14.1 Makefile

```
all: hello

hello: hello.o main.o
    g++ -o hello hello.o main.o

hello.o: hello.cpp
    g++ -o hello.o -c hello.cpp -W -Wall -ansi -pedantic

main.o: main.cpp hello.h
    g++ -o main.o -c main.cpp -W -Wall -ansi -pedantic

clean:
    rm -rf *.o

mrproper: clean
    rm -rf hello
```

15 Variables

Une variable se déclare sous la forme `NOM=VALEUR` et est utilisée via `$(NOM)`.

15.0.0.1 Variables personnalisées

Les variables personnalisées les plus courantes sont :

- `CXX` : le compilateur C++ (CC pour un compilateur C).
- `CXXFLAGS` : les options de compilation en C++ (CFLAGS pour une compilation en C).
- `LDFLAGS` : les options de l'édition de liens.
- `EXEC` : le nom des exécutables à générer.

Le Makefile devient ainsi :

15.0.0.2 Variables prédéfinies

Certaines variables sont prédéfinies :

Variable	Signification
<code>\$@</code>	Le nom de la cible
<code>\$<</code>	Le nom de la première dépendance
<code>\$^</code>	La liste des dépendances
<code>\$?</code>	La liste des dépendances plus récentes que la cible
<code>\$*</code>	Le nom du fichier sans suffixe

Le Makefile ainsi devient :

i Note

Demo

Listing 15.1 Makefile

```
CC=g++
CFLAGS=-W -Wall -ansi -pedantic
LDFLAGS=
EXEC=hello

all: $(EXEC)

hello: hello.o main.o
    $(CC) -o hello hello.o main.o $(LDFLAGS)

hello.o: hello.c
    $(CC) -o hello.o -c hello.c $(CFLAGS)

main.o: main.c hello.h
    $(CC) -o main.o -c main.c $(CFLAGS)

clean:
    rm -rf *.o

mrproper: clean
    rm -rf $(EXEC)
```

Listing 15.2 Makefile

```
CXX=g++
CXXFLAGS=-W -Wall -ansi -pedantic
LDFLAGS=
EXEC=hello

all: $(EXEC)

hello: hello.o main.o
    $(CXX) -o $@ $^ $(LDFLAGS)

hello.o: hello.c
    $(CXX) -o $@ -c $< $(CXXFLAGS)

main.o: main.c hello.h
    $(CXX) -o $@ -c $< $(CXXFLAGS)

clean:
    rm -rf *.o

mrproper: clean
    rm -rf $(EXEC)
```

16 Cmake

CMake est un outil qui permet de générer un Makefile, compiler un projet pour différentes plateformes, faire des tests et créer des packages pour différents systèmes.

16.0.1 Autre

Les projets CMake se reconnaissent grâce à la présence du fichier `CMakeLists.txt` (généralement à la racine du projet).

Le `CMakeLists.txt` doit décrire votre projet. Pour cela, il faudra lui indiquer les éléments suivants :

- la version minimale pour lire le fichier `CMakeLists.txt` (`cmake_minimum_required()`) ;
- le nom du projet (`project()`) ;
- les dépendances à ajouter (`target_include_directories()/target_link_libraries()`) ;
- le binaire à produire (`add_executable()/add_library()`).

`CMakeLists.txt`

```
cmake_minimum_required(VERSION 3.10)
project(CalculatorProject)

set(CMAKE_CXX_STANDARD 17)

include_directories(include)
add_library(calculator src/Calculator.cpp)
```

Pour compiler et exécuter le code

```
mkdir build
cd build
cmake ..
make
./test_calculator
```